```
#Machine Learning Mastery Bootcamp - DevTown
#Final Machine Learning Project

#Name: Kaveen Amarasekara
#Email: kaveenamarasekara2@gmail.com
#College: University of Colombo
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
```

```
df = pd.read_csv('customer_retail_FINAL.csv')

df.head()
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	01-12-2010 08:26	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	01-12-2010 08:26	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	01-12-2010 08:26	2.75	17850.0	United Kingdom
3	536365	84029G	KNITTED UNION FLAG HOT WATER BOTTLE	6	01-12-2010 08:26	3.39	17850.0	United Kingdom
4	536365	84029E	RED WOOLLY HOTTIE WHITE HEART.	6	01-12-2010 08:26	3.39	17850.0	United Kingdom

Next steps: Generate code with df | New interactive sheet

```
df.info() # Dataset info

df.isnull().sum() # Check missing values
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 24721 entries, 0 to 24720
Data columns (total 8 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   InvoiceNo    24721 non-null  object
 1   StockCode    24721 non-null  object
 2   Description  24610 non-null  object
 3   Quantity     24721 non-null  int64
 4   InvoiceDate  24721 non-null  object
 5   UnitPrice    24721 non-null  float64
 6   CustomerID   16030 non-null  float64
 7   Country      24721 non-null  object
dtypes: float64(2), int64(1), object(5)
memory usage: 1.5+ MB
```

	0
InvoiceNo	0
StockCode	0
Description	111
Quantity	0
InvoiceDate	0
UnitPrice	0
CustomerID	8691
Country	0

dtype: int64

```
# Remove rows with missing values
df = df.dropna()

df.isnull().sum()
```

	0
InvoiceNo	0
StockCode	0
Description	0
Quantity	0
InvoiceDate	0
UnitPrice	0
CustomerID	0
Country	0

dtype: int64

```
df = df[['Quantity', 'UnitPrice', 'Country']]

df.head()
```

	Quantity	UnitPrice	Country
0	6	2.55	United Kingdom
1	6	3.39	United Kingdom
2	8	2.75	United Kingdom
3	6	3.39	United Kingdom
4	6	3.39	United Kingdom

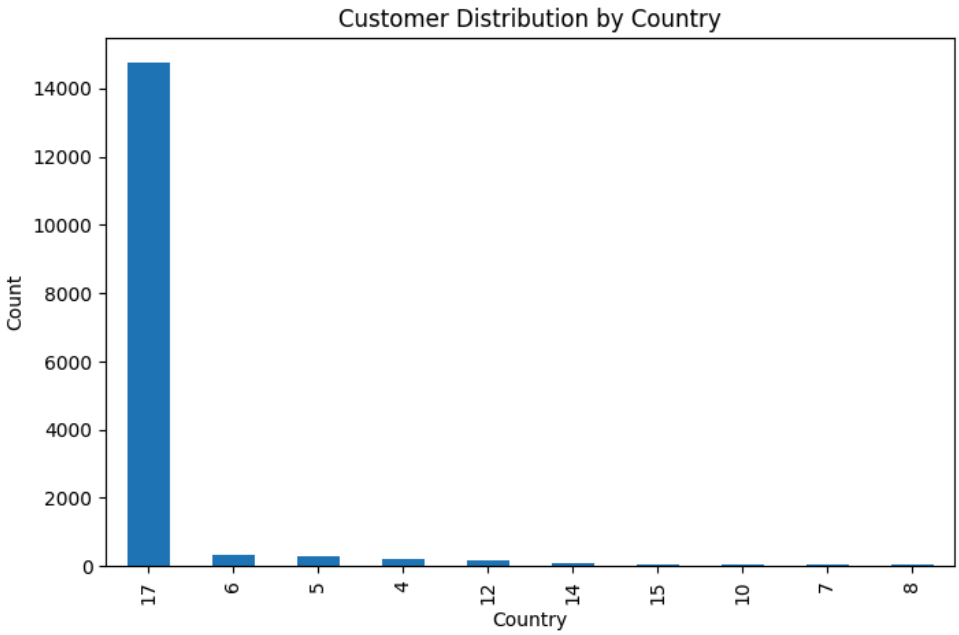
Next steps: Generate code with df | New interactive sheet

```
encoder = LabelEncoder()

df['Country'] = encoder.fit_transform(df['Country'])

df.head()
```

	Quantity	UnitPrice	Country
0	6	2.55	17
1	6	3.39	17
2	8	2.75	17
3	6	3.39	17
4	6	3.39	17

Next steps: Generate code with `df` New interactive sheet

```
# Customer distribution graph

plt.figure(figsize=(8,5))

df['Country'].value_counts().head(10).plot(kind='bar')

plt.title('Customer Distribution by Country')
plt.xlabel('Country')
plt.ylabel('Count')

plt.show()
```




```
# Features (inputs)
X = df[['Quantity', 'UnitPrice']]

# Target (output)
y = df['Country']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

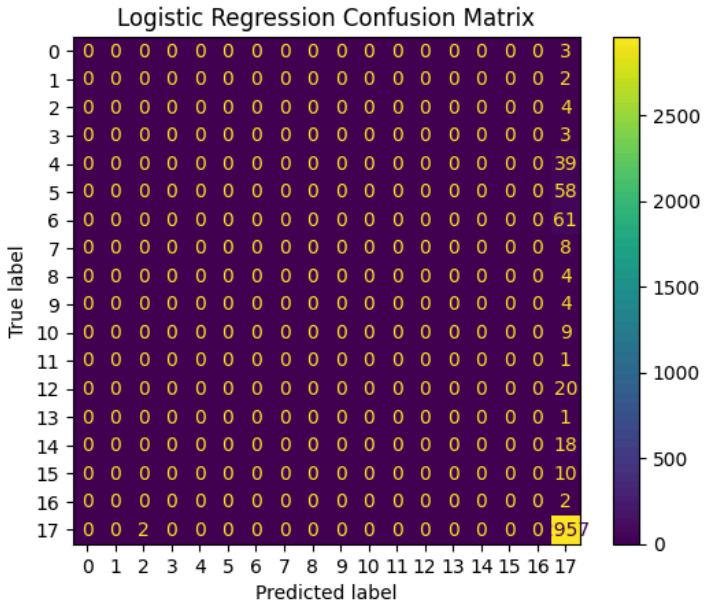
```
Training data: (12824, 2)
Testing data: (3206, 2)
```

```
# Logistic Regression model

lr_model = LogisticRegression(max_iter=1000)

lr_model.fit(X_train, y_train)


```

```
lr_pred = lr_model.predict(X_test)


lr_accuracy = accuracy_score(y_test, lr_pred)

print("Logistic Regression Accuracy:", lr_accuracy)
```

```
Logistic Regression Accuracy: 0.922333125389894
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to conve
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

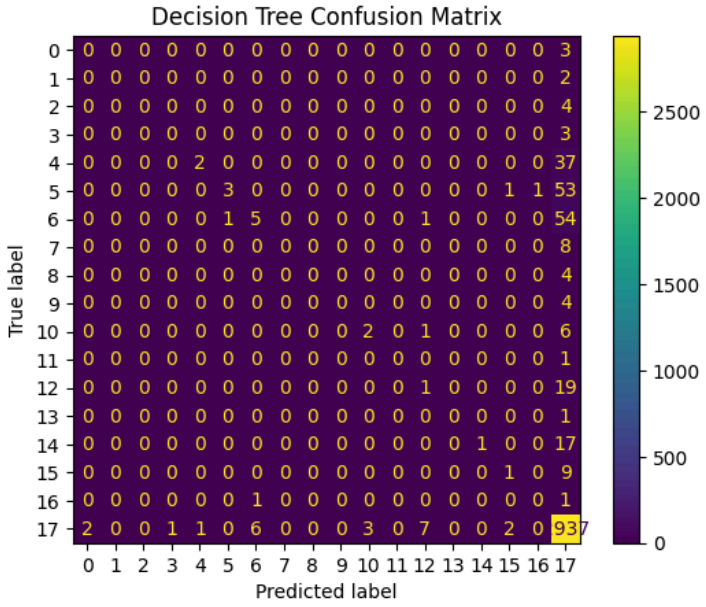
```
# Confusion Matrix LR

cm = confusion_matrix(y_test, lr_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)

disp.plot()

plt.title("Logistic Regression Confusion Matrix")

plt.show()
```




```
# Decision Tree model

dt_model = DecisionTreeClassifier()

dt_model.fit(X_train, y_train)

dt_pred = dt_model.predict(X_test)

dt_accuracy = accuracy_score(y_test, dt_pred)

print("Decision Tree Accuracy:", dt_accuracy)
```

```
Decision Tree Accuracy: 0.9207735495945103
```

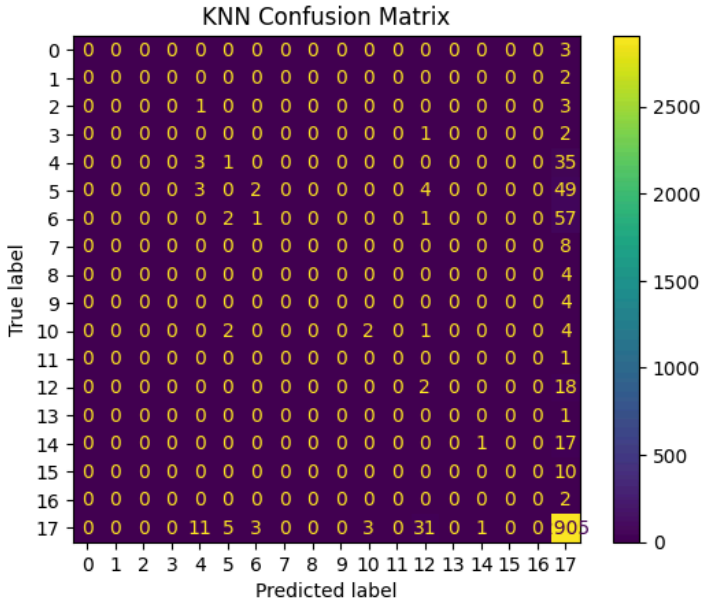
```
# Confusion Matrix DT

cm = confusion_matrix(y_test, dt_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)

disp.plot()

plt.title("Decision Tree Confusion Matrix")

plt.show()
```




```
# KNN model

knn_model = KNeighborsClassifier(n_neighbors=5)

knn_model.fit(X_train, y_train)

knn_pred = knn_model.predict(X_test)


knn_accuracy = accuracy_score(y_test, knn_pred)

print("KNN Accuracy:", knn_accuracy)
```

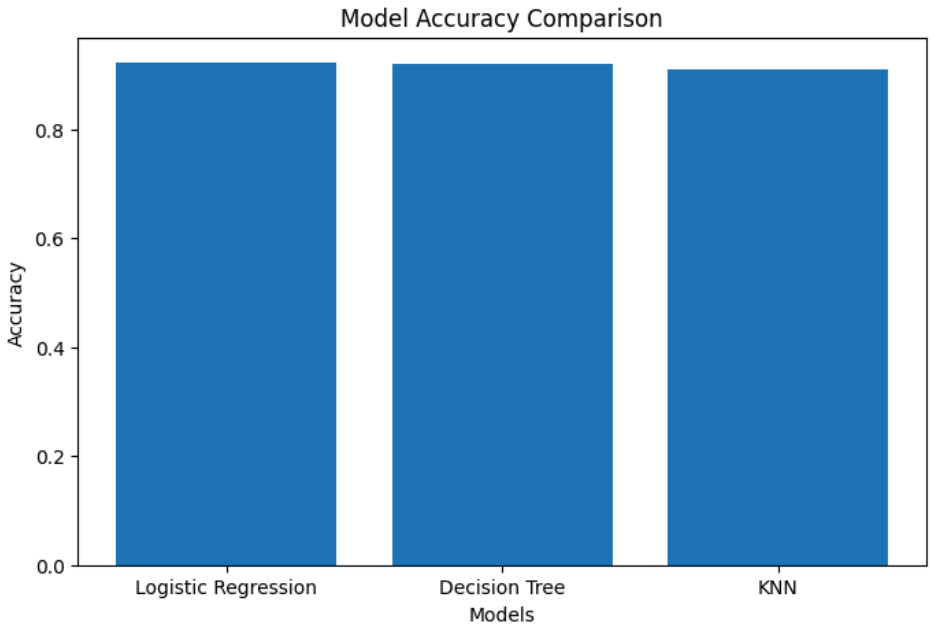
```
KNN Accuracy: 0.9089207735495946
```

```
# Confusion Matrix KNN

cm = confusion_matrix(y_test, knn_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm)

disp.plot()

plt.title("KNN Confusion Matrix")

plt.show()
```




```
# Model comparison graph
```

```
models = ['Logistic Regression', 'Decision Tree', 'KNN']

accuracies = [
    lr_accuracy,
    dt_accuracy,
    knn_accuracy
]

plt.figure(figsize=(8,5))

plt.bar(models, accuracies)

plt.title('Model Accuracy Comparison')
plt.xlabel('Models')
plt.ylabel('Accuracy')

plt.show()
```




```
print("Final Accuracy Results")
print("-----------------------")

print("Logistic Regression: ", lr_accuracy)
```